



Apache Cassandra

Claudia Cornacchia
Andrea Lanzarone



Outline

1

Motivation

2

Data model

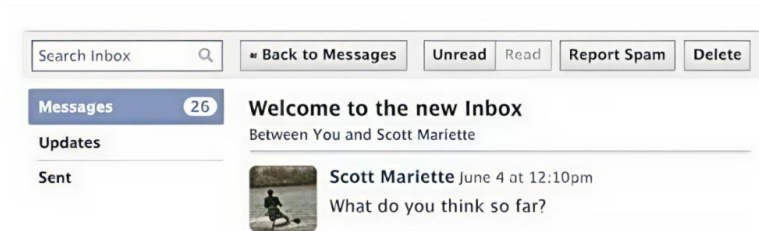
3

Distributed architecture

Motivations behind Cassandra



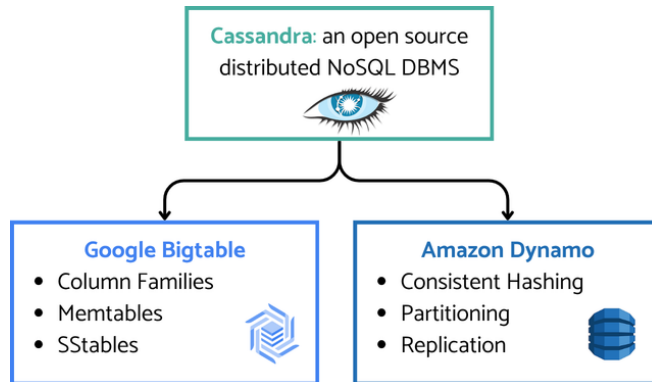
Facebook Inbox Search: Enabling real-time search across billions of Facebook private messages.



Challenges:

- **Massive write workloads:** to make messages searchable, need to index every word
- **Exponential growth:** continuous growth, need to scale horizontally across hundred of commodity hardware
- **Failure as the norm:** need to guarantee fault tolerance without SPOF
- **Global user distribution:** ensure low latency for user spread worldwide by replicating data

Solution:

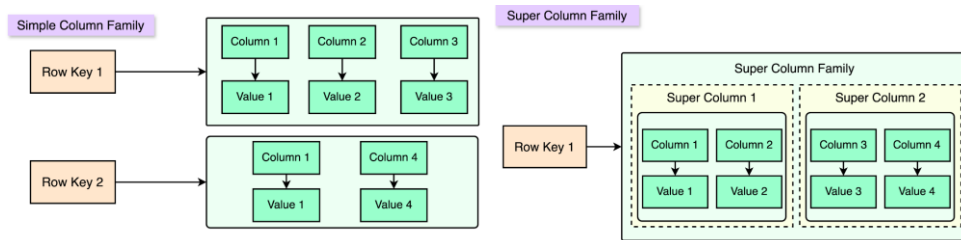


Data Model

Column-Family data model: derived from Google BigTable

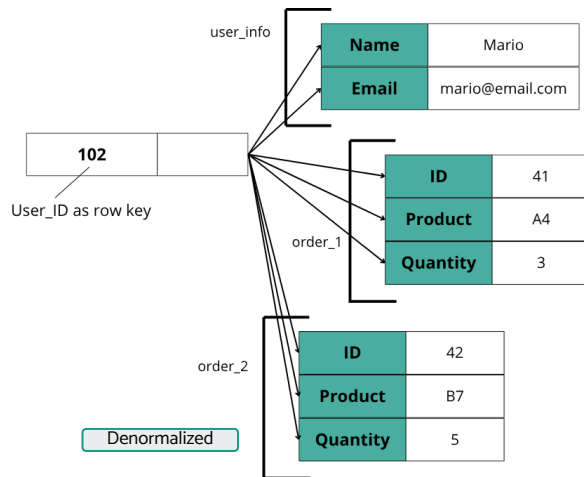
Data is organized as a two dimensional map:

- **Row key** identifies the aggregate
- **Columns** stores key-value pairs grouped into column or supercolumn families



Query-Driven approach:

- Data is **denormalized** into a single aggregate
- No need for JOINS
- Efficient horizontal scaling



Relational model (Data-Driven):

- Related data spread across multiple tables
- Requires costly cross-server JOINS that increase latency and forces Scale-Up

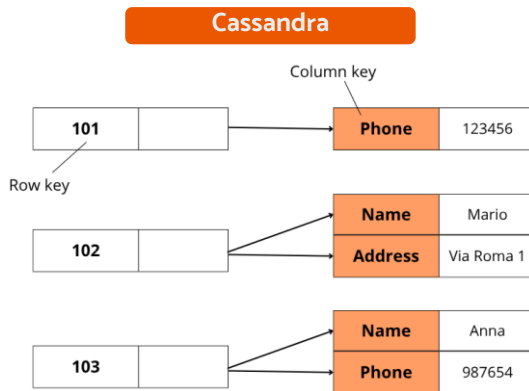
User Table			Order Table			
User_ID	Name	Email	Order_ID	User_ID	Product	Quantity
101	Luca	luca@email.com	41	102	A4	3
102	Mario	mario@email.com	42	102	B7	5
103	Anna	anna@email.com	43	101	A2	7

Normalized

Data Model

Schemaless paradigm:

- **Row-Level flexibility:** Rows within the same family can have completely different sets of columns.
- **Avoid sparse data:** Eliminates wasted space caused by NULL values (typical of rigid relational databases)
- No rigid schema → cannot enforce integrity constraints

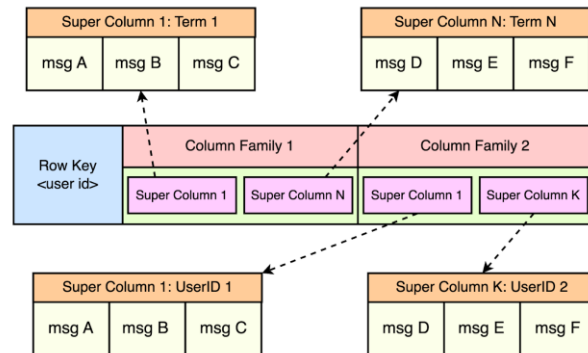


Relational

User_ID	Name	Phone	Address
101	NULL	123456	NULL
102	Mario	NULL	Via Roma 1
103	Anna	987654	NULL

This design efficiently solves the Facebook Inbox Search problem:

- **Row Key:** User ID
- **Super Columns:** words (Term search) or User ID (search for messages exchanged with a specific user)
- **Sub-Columns:** message IDs



Data Model: comparison with other NoSQL

- **Key-Value**: aggregate as an **opaque** blob that must be read entirely. Good for simple lookups.
- **Document-based**: makes the aggregate **transparent**. Allows for internal querying, but due to the absence of rigid schema, sacrifices physical storage optimization.
- **Column-Family** (Cassandra): the Middle Ground. Enforces enough structure to optimize storage and write efficiency, while allowing column-level access.

Key-Value

Key	Value
employee_1	name@Tom-surn@Smith-off@41-buil@A4-tel@45798
employee_2	name@John-surn@Doe-off@42-buil@B7-tel@12349
employee_3	name@Tom-surn@Smith
office_41	buil@A4-tel@45798
office_42	buil@B7-tel@12349

Document-based

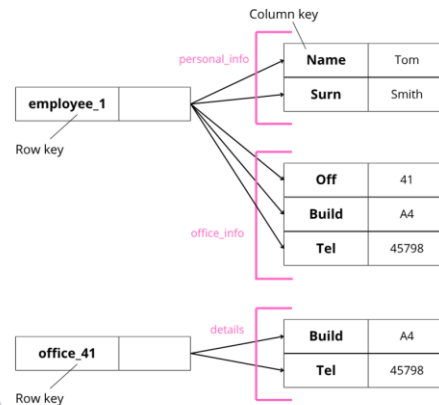
Key: "employee_1" →

```
{
  id: "1" .
  name: "Tom" .
  surname: "Smith" .
  office: {
    id: "41" .
    building: "A4" .
    telephone: "45798"
  }
}
```

Key: "office_41" →

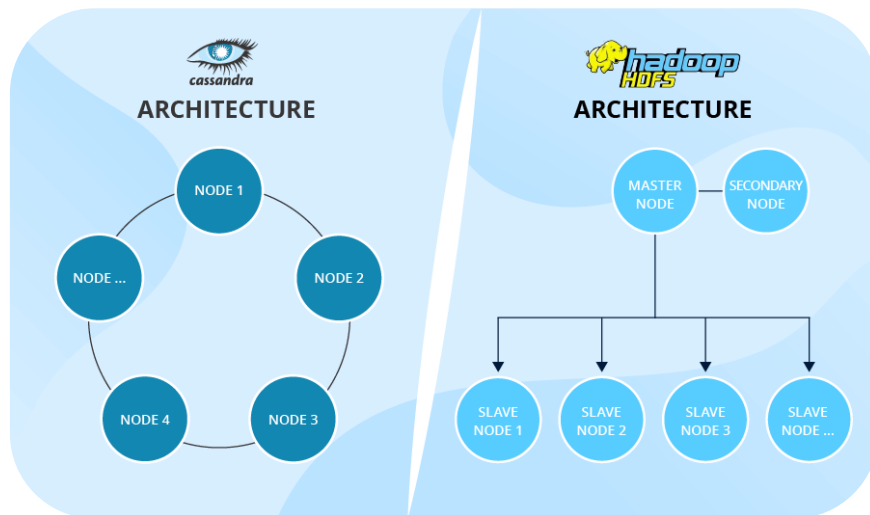
```
{
  id: "41" .
  building: "A4" .
  telephone: "45798"
}
```

Column-Family



Distributed Architecture: Peer-to-Peer

- Fully **decentralized, peer-to-peer** architecture. Nodes can handle read and write independently.
- Unlike Master-Slave systems (HDFS or MongoDB), it has **no central master**.
- **No Single Point of failure**, cluster remains available even if individual nodes fail.



Distributed Architecture: Data partitioning

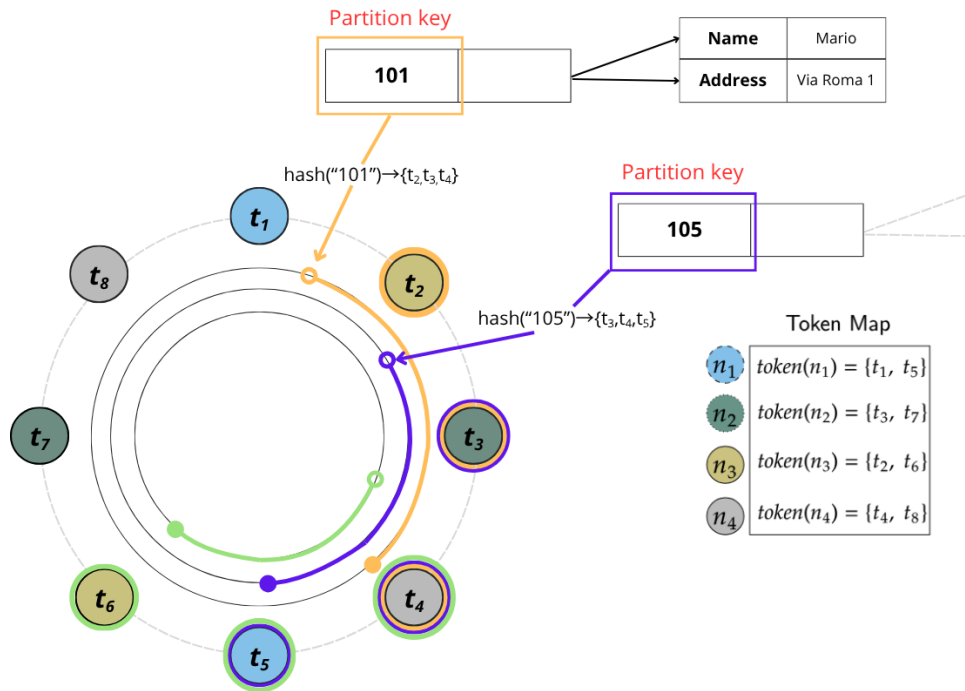
Achieving horizontal scalability

- Distributes partitions using **consistent hashing** across a continuous token ring (Dynamo). While ensuring all the data of an entire aggregate are stored together.

Balanced load:

- Physical servers acts as virtual nodes, own multiple tokens on the ring.
- When a node is added, its token are placed in the heaviest sections to ensure a balanced load while keeping rest of cluster unaffected → solves the data redistribution problem of naive hashing.

Routing Mechanism: the coordinator hashes the partition key and then walks clockwise to find the first assigned token.



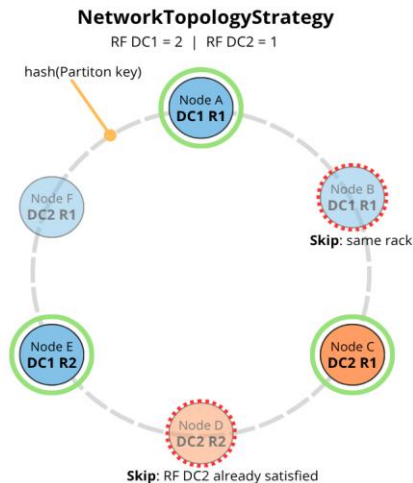
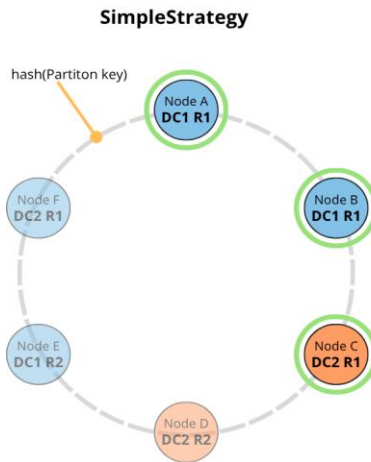
Distributed Architecture: Data replication

Achieving high availability and fault tolerance

Routing Mechanism: the coordinator node then replicates data to the next N-1 distinct physical nodes (N Replication Factor).

Replication strategy

- SimpleStrategy: places replicas on the next N-1 distinct physical nodes regardless of the network topology
- NetworkTopologyStrategy:
 - Distinct RF for each datacenter
 - Within each datacenter place replicas on different physical racks



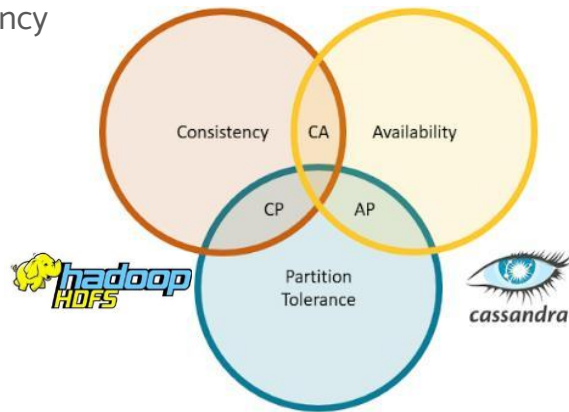
Distributed Architecture: Eventual Consistency

- **AP System:** Prioritizes Availability and Partition Tolerance over Strong Consistency



Eventual consistency

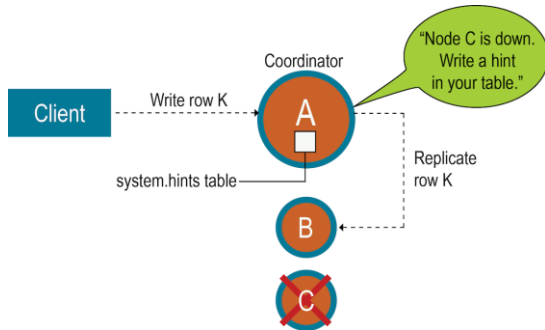
- Unlike CP systems like HDFS that wait for every replica to save the data before confirming the write Cassandra is optimized for high-speed operations
- **Tunable Consistency:** Balance consistency and availability per operation



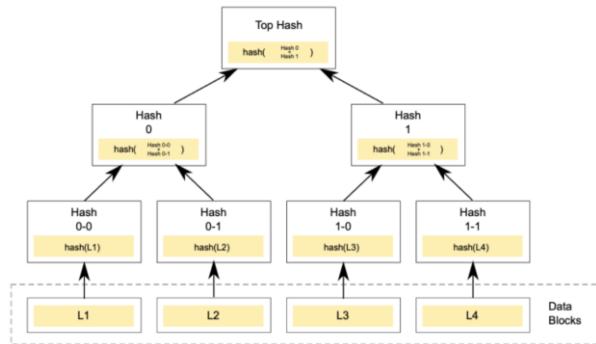
Distributed Architecture: Replica synchronization mechanism

Best effort :

- **Read Repair**: update replicas with older timestamp detected during reads.
- **Hinted Handoff**: the coordinator saves a temporary 'hint' and delivers it as soon as the node comes back online.



- **Anti-Entropy Repair**: Instead of scanning entire datasets, servers simply compare the root hash for a shared key range.



Improved Dynamo design:

- Sub-range repair
- Incremental repair

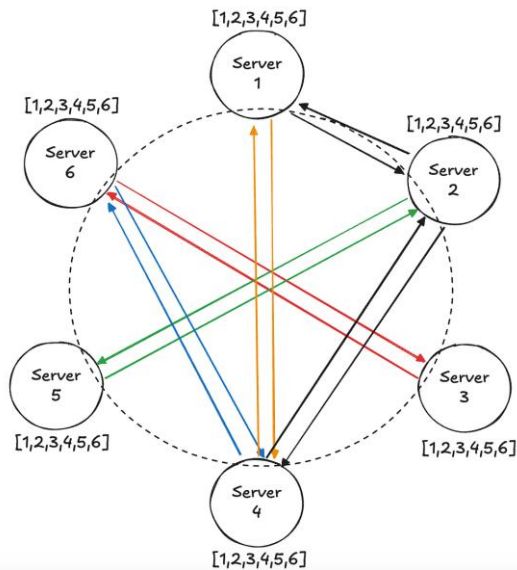
Distributed Architecture: Gossip protocol and Failure detection

Gossip Protocol (Scuttlebutt): Nodes periodically exchange information like heartbeats, schema changes with their neighbors.

Φ (Phi) Accrual Failure Detector: Locally on every node. Instead of rigid timeout, nodes evaluate heartbeat delays to update a suspicious level that adapts to network and load condition.

Seed Nodes:

- Initial state information for new nodes
- Propagate the new node's presence across the system



High Write Throughput: The Storage Engine

1. A write occurs

2. Appends the data sequentially into the Commit Log

3. Data is placed in memtable where is sorted by key

4. Memtable fills it is flushed to immutable SSTable

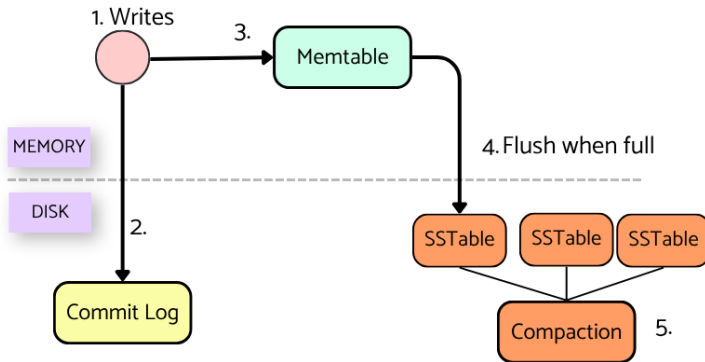
5. Background process called Compaction merges these multiple internally sorted SSTable files into larger ones, it uses a RAM-efficient merge strategy

Durability and recoverability

Faster than writing and sorting directly in storage

High write throughput: Avoids performing slow in-place updates with heavy locking like traditional B-Tree databases

→ Conflicts across versions are resolved automatically during reads using Last-Write-Wins (unlike Dynamo)





Demo!



References

- **Official Cassandra Paper:** Lakshman, A., Malik, P. (2010). "Cassandra: a decentralized structured storage system."
- **Apache Cassandra Official Documentation:** <https://cassandra.apache.org/doc/latest/>
- **Big Data Course material**
- **Dynamo:** DeCandia, G., et al. (2007). "Dynamo: Amazon's Highly Available Key-value Store."